

class07

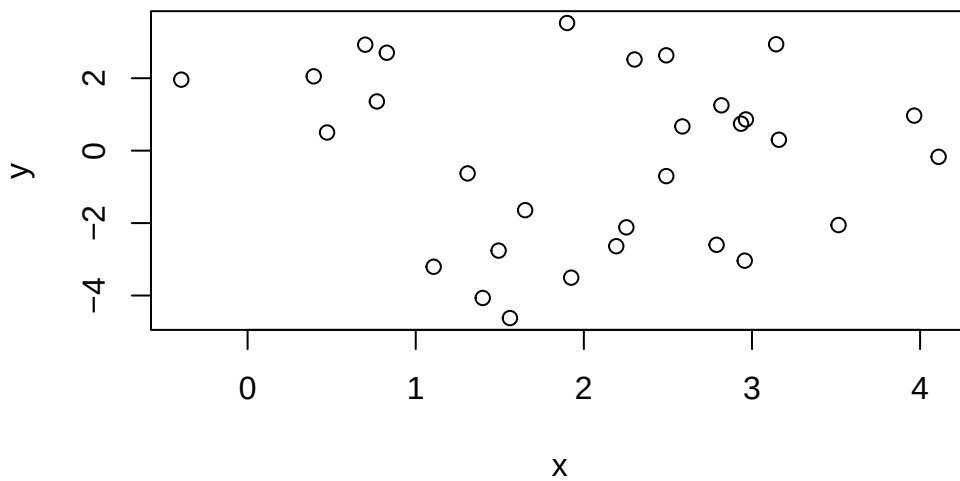
Nicolas Song

Generate some data

```
tmp_x <- c(rnorm(15,2), rnorm(15,2))  
tmp_y <- c(rnorm(15,2), rnorm(15,-2))  
  
df <- cbind(x=tmp_x, y=tmp_y)  
  
View(df)
```

Plot the data

```
plot(df)
```



Let's do a k-means clustering analysis

```
km <- kmeans(df, 2, nstart=20)
```

```
km
```

K-means clustering with 2 clusters of sizes 17, 13

Cluster means:

	x	y
1	2.067189	1.63151
2	2.049242	-2.58353

Clustering vector:

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

Within cluster sum of squares by cluster:

```
[1] 48.72454 22.93875  
(between_SS / total_SS = 64.6 %)
```

Available components:

[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

```
attributes(km)
```

\$names

[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

\$class

```
[1] "kmeans"
```

Q: Number of points per cluster?

```
km$size
```

```
[1] 17 13
```

Q: Cluster size? `km$size`

Q: Cluster assignment

```
km$cluster
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

```
table(km$cluster)
```

```
1  2  
17 13
```

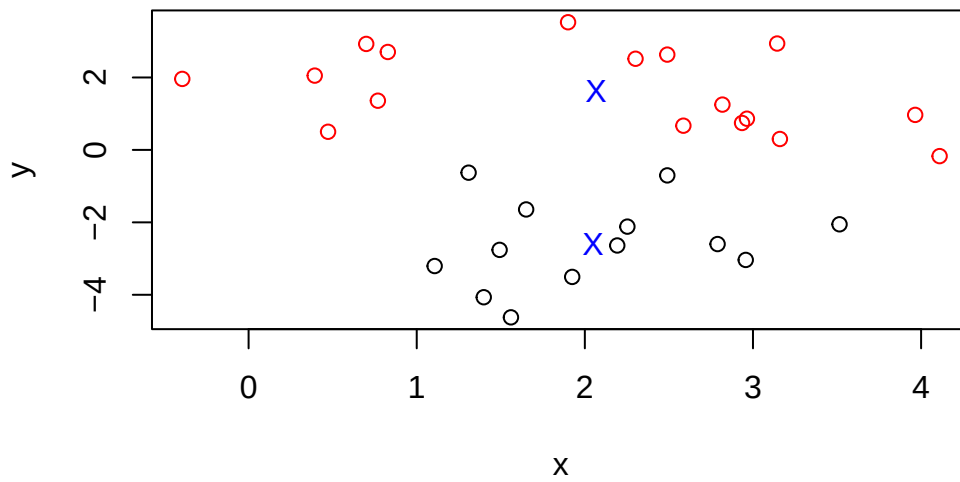
Q: Cluster? centers

```
km$centers
```

```
      x      y  
1 2.067189 1.63151  
2 2.049242 -2.58353
```

Plot k-means clustering:

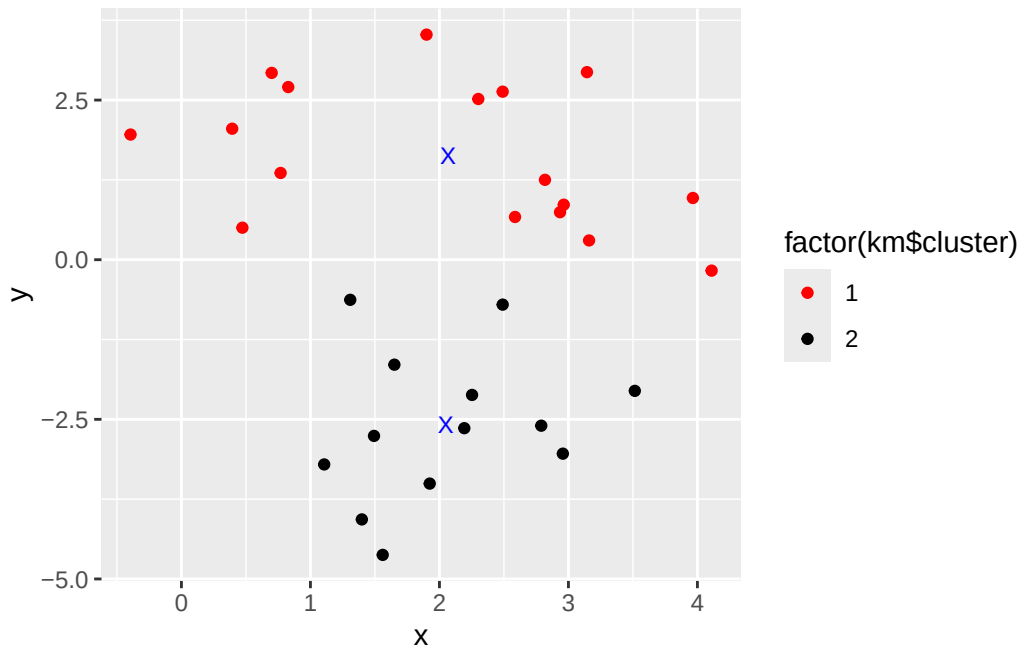
```
plot(df, col=c("RED", "BLACK")[km$cluster])  
points(km$centers, col="BLUE", pch="X")
```



Plot with ggplot:

```
library(ggplot2)

ggplot(df) +
  aes(x, y, col=factor(km$cluster)) +
  geom_point() +
  scale_color_manual(values=c("red", "black")) +
  geom_point(data=km$centers, col="blue", shape="X",
             size=3, aes(x, y))
```



Hierarchical clustering

Analyze by hclust

```
dm <- dist(df)
```

```
#dm
```

```
hc <- hclust(dm)
```

```
hc
```

Call:

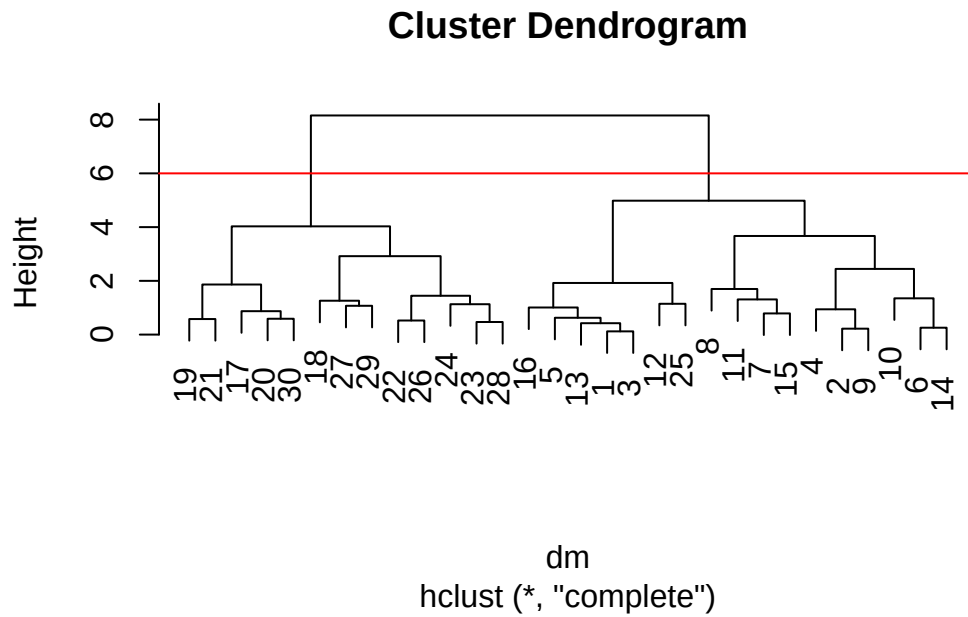
```
hclust(d = dm)
```

Cluster method : complete

Distance : euclidean

Number of objects: 30

```
plot(hc)
abline(h=6, col="red")
```



Let's use cutree:

Cut by height

```
table(cutree(hc, h=6))
```

```
1 2
17 13
```

Cut by k

```
cutree(hc, k=2)
```

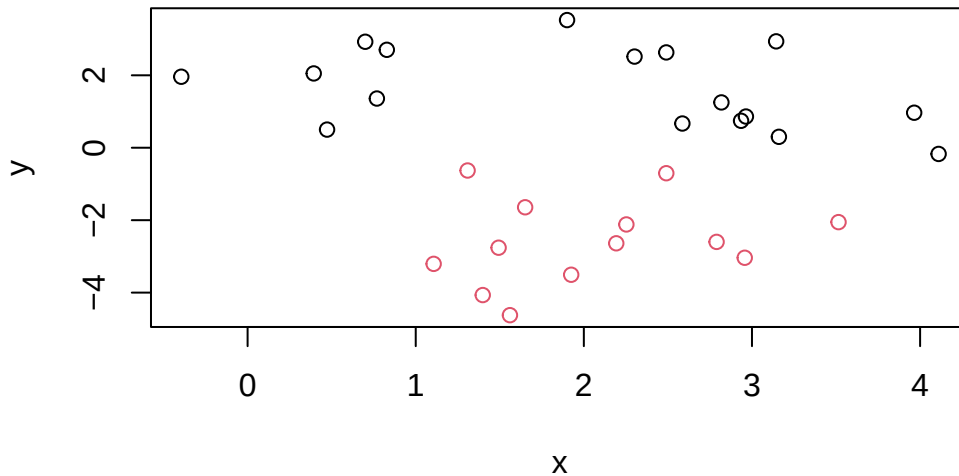
```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 1 2 2 2 2 2
```

4 clusters

```
cutree(hc, k=4)
```

```
[1] 1 2 1 2 1 2 2 2 2 2 2 1 1 2 2 1 3 4 3 3 3 4 4 4 1 4 4 4 4 3
```

```
plot(df, col=cutree(hc, k=2))
```



1. PCA of UK food data

```
url <- "https://tinyurl.com/UK-foods"  
x <- read.csv(url)
```

Q1: How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
# dim() used to find the number of rows and columns in x  
dim(x)
```

```
[1] 17  5
```

Q2 setup

```
# removing the first column labeled "X"
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
# checking the new dimensions of the data frame
dim(x)
```

```
[1] 17  4
```

```
# Alternative approach to setting the correct row-names
x <- read.csv(url, row.names=1)
head(x)
```

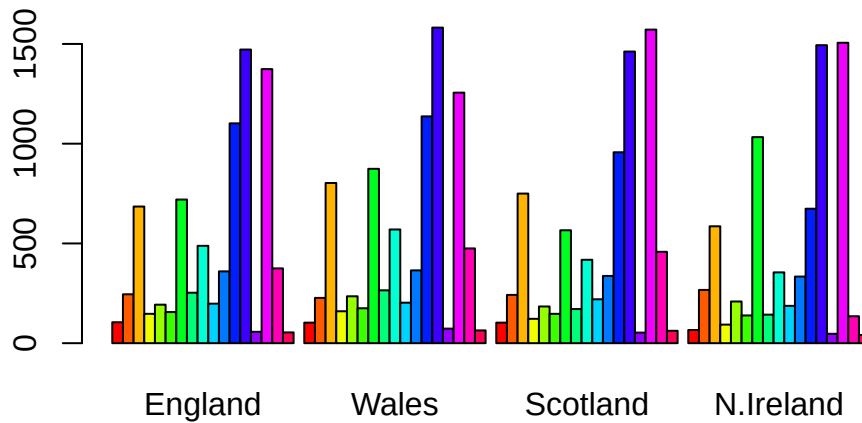
	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

Q2: Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the alternative one because it requires much less code than the first. The original approach seems like it would be more robust for a variety of circumstances because it allows you to single out a column, no matter where it is on the data set.

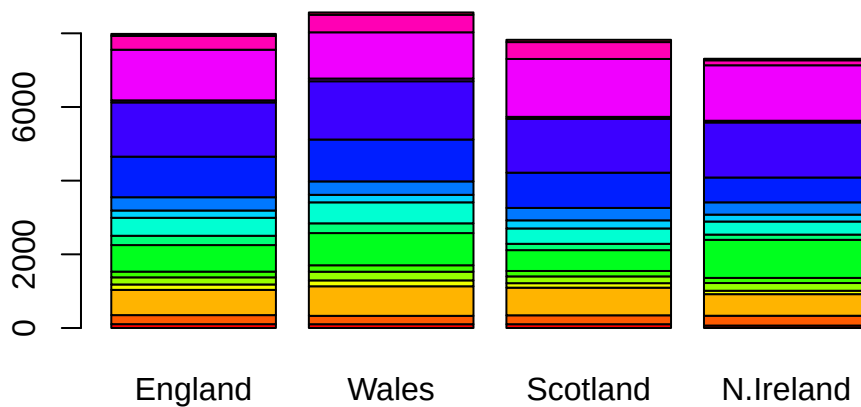
Spotting major differences and trends

```
# Using base R  
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

```
# Changing beside to false to achieve a stacked plot  
barplot(as.matrix(x), beside=F, col=rainbow(nrow(x)))
```



```
library("tidyr")

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
  tibble::rownames_to_column("Food") |>
  pivot_longer(cols = -Food,
               names_to = "Country",
               values_to = "Consumption")

dim(x_long)
```

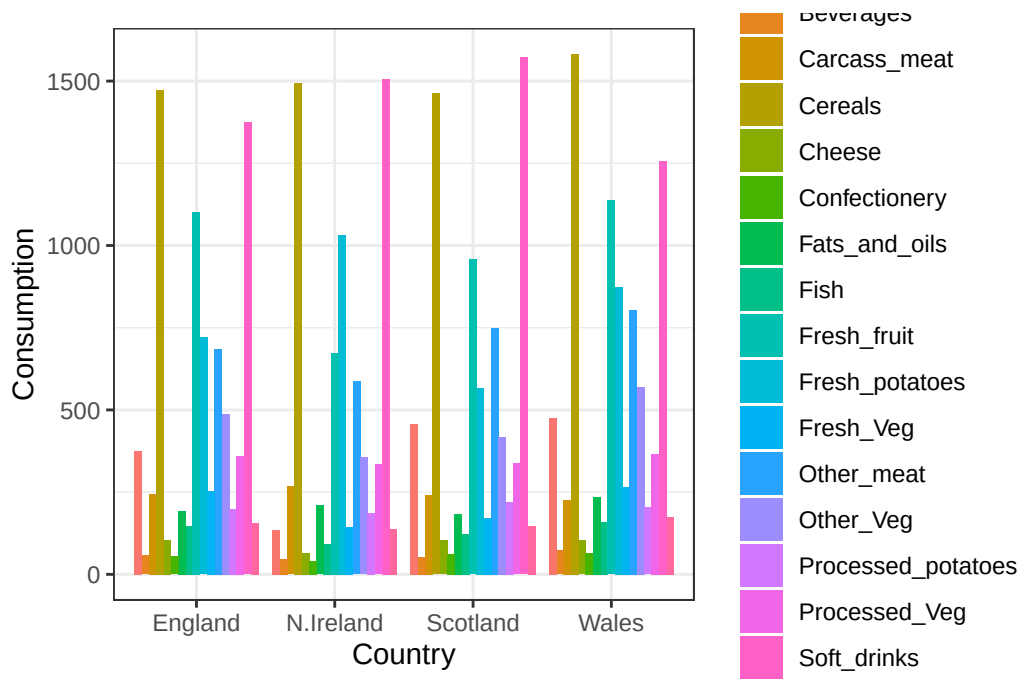
```
[1] 68  3
```

```
head(x_long)
```

```
# A tibble: 6 x 3
  Food      Country Consumption
<chr>    <chr>         <int>
1 "Cheese" England         105
2 "Cheese" Wales           103
3 "Cheese" Scotland         103
```

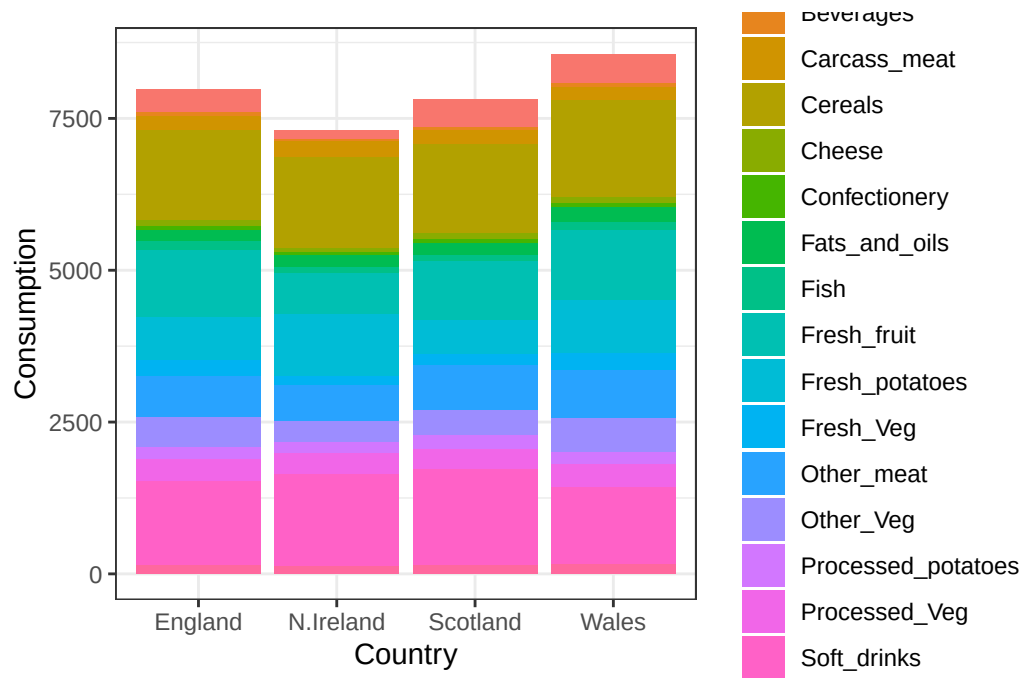
4	"Cheese"	N.Ireland	66
5	"Carcass_meat "	England	245
6	"Carcass_meat "	Wales	227

```
# Create grouped bar plot
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) + geom_col(position = "dodge") +
  theme_bw()
```



Q4: Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

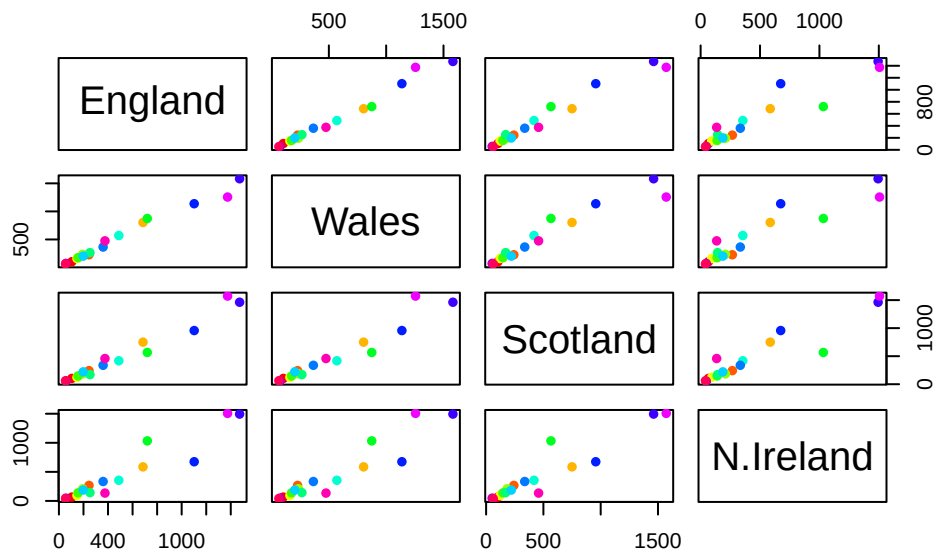
```
# Changing position from "dodge" to "stack"
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) + geom_col(position = "stack") +
  theme_bw()
```



Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

If a given point lies on the diagonal for a given plot, it means that the point's value is identical to that of the one it is being compared to in the pairwise assessment.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```

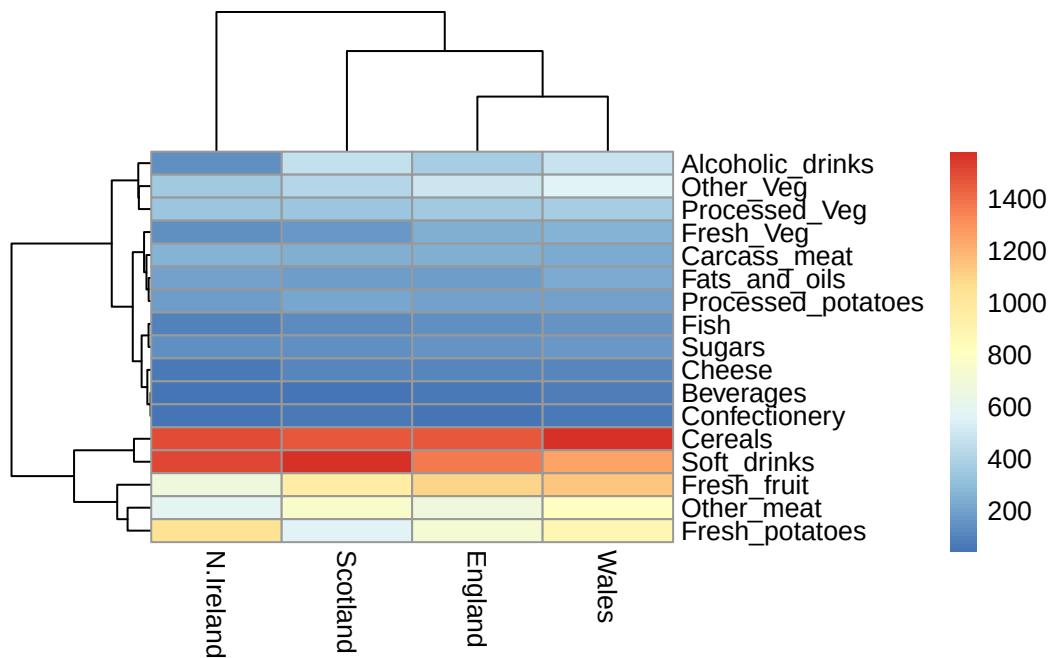


Q6: Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Wales and England cluster together and with Scotland, while North Ireland seems to be the least clustered with the rest of the countries in the UK, though it is pretty difficult to tell what the main differences are based on this figure.

```
# Generating a heatmap
library(pheatmap)

pheatmap( as.matrix(x) )
```



PCA to the rescue

```
# Use the prcomp() PCA function
pca <- prcomp( t(x) )
summary(pca)
```

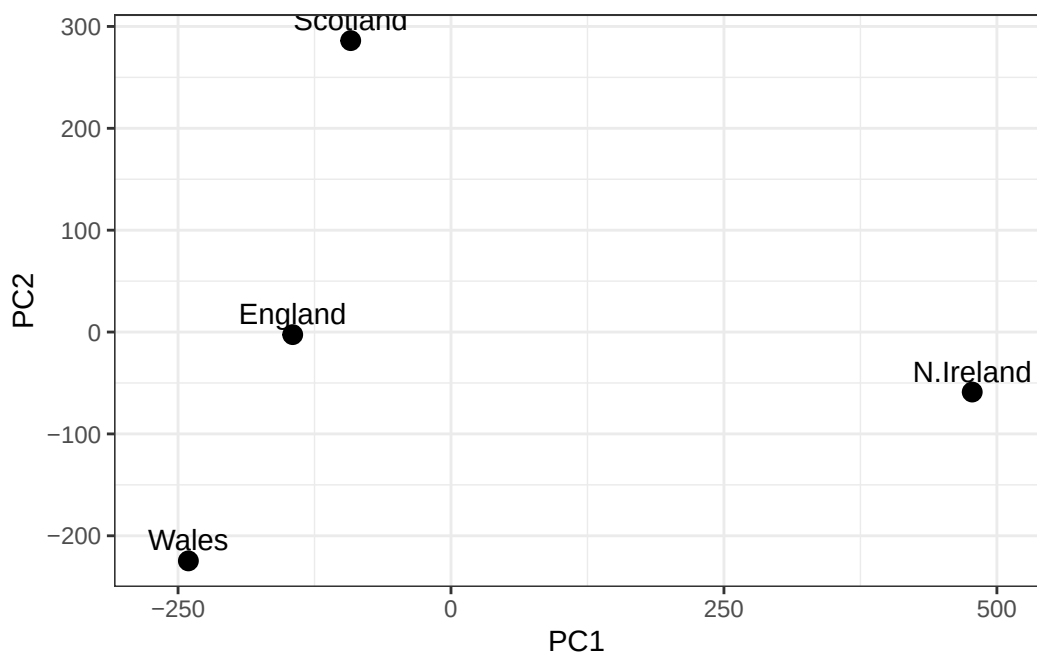
Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.921e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.000e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.000e+00

Q7: Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)
```

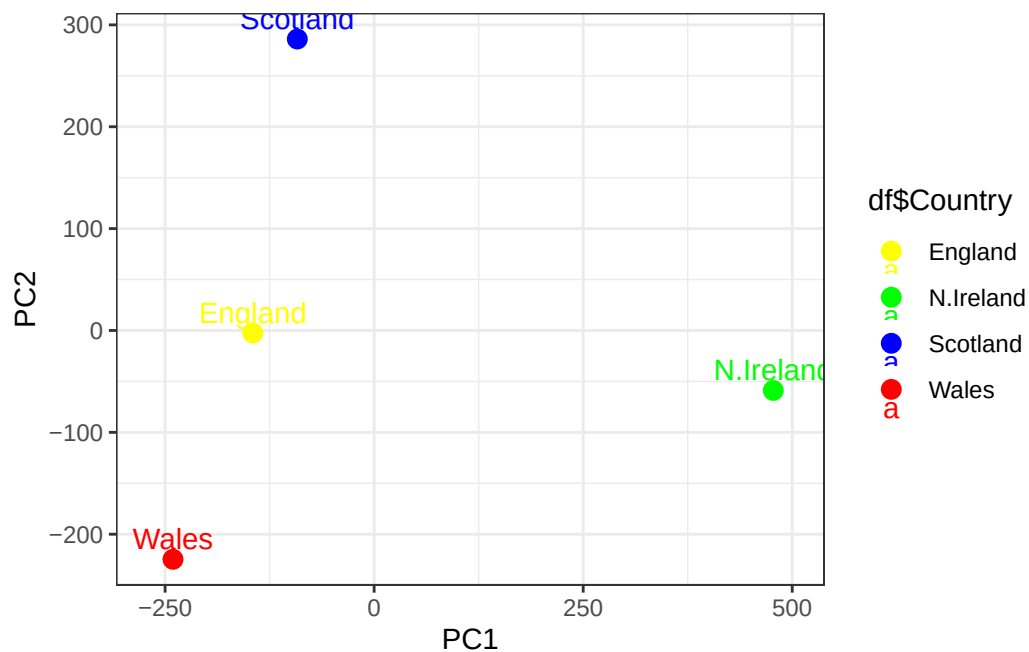
```
# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Q8: Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.

```
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x),
    # coloring by country
    color=df$Country) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw() +
```

```
# assigning colors
scale_color_manual(values=c("yellow","green", "blue","red"))
```



```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

```
[1] 67 29 4 0
```

```
z <- summary(pca)
z$importance
```

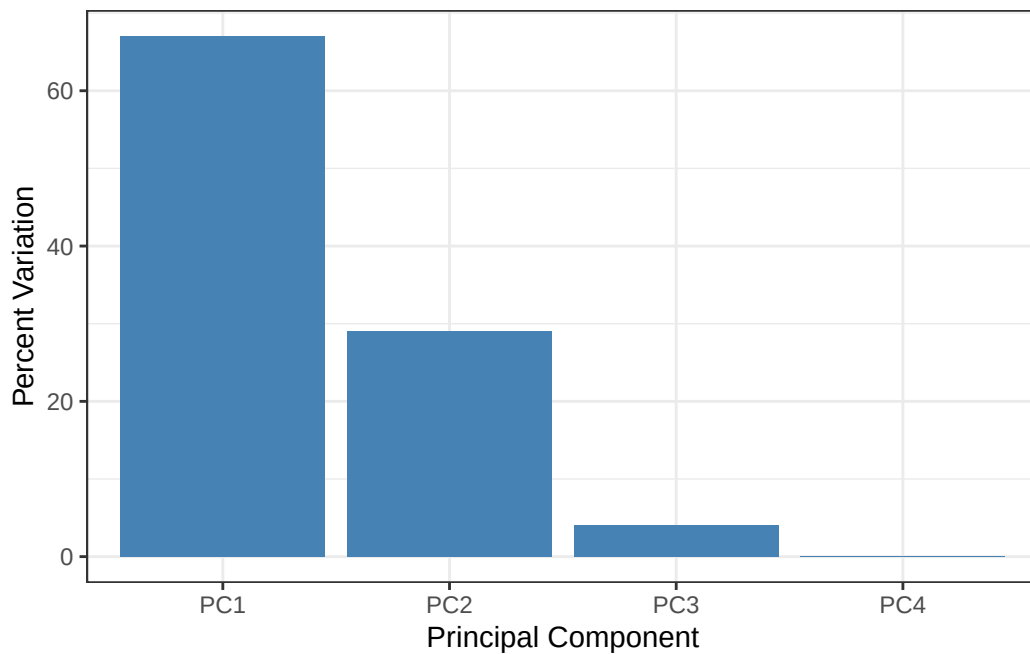
	PC1	PC2	PC3	PC4
Standard deviation	324.15019	212.74780	73.87622	2.921348e-14
Proportion of Variance	0.67444	0.29052	0.03503	0.000000e+00
Cumulative Proportion	0.67444	0.96497	1.00000	1.000000e+00

```
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
```

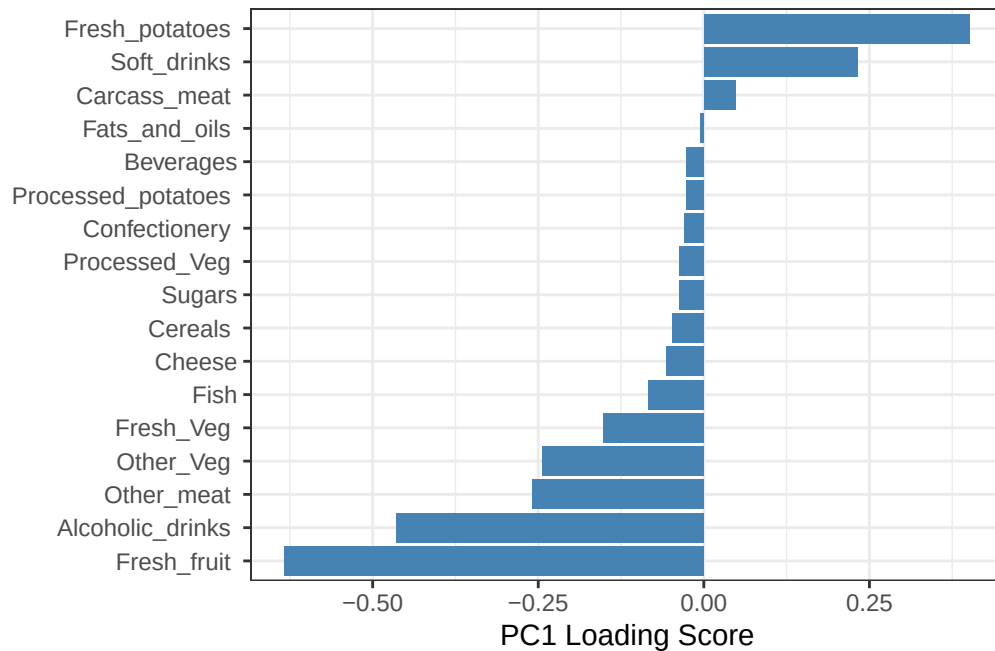


```
)
```

```
ggplot(variance_df) +  
  aes(x = PC, y = Variance) +  
  geom_col(fill = "steelblue") +  
  xlab("Principal Component") +  
  ylab("Percent Variation") +  
  theme_bw() +  
  theme(axis.text.x = element_text(angle = 0))
```



```
## Lets focus on PC1 as it accounts for > 90% of variance  
ggplot(pca$rotation) +  
  aes(x = PC1,  
      y = reorder(rownames(pca$rotation), PC1)) +  
  geom_col(fill = "steelblue") +  
  xlab("PC1 Loading Score") +  
  ylab("") +  
  theme_bw() +  
  theme(axis.text.y = element_text(size = 9))
```



Q9: Generate a similar ‘loadings plot’ for PC2. What two food groups feature prominently and what does PC2 mainly tell us about?

The largest positive loading score is Soft_drinks and the largest negative score is Fresh_potatoes. PC2 thus tells us that when consumption of soft drinks increases, consumption of fresh potatoes decreases.

```
# PC2 loading plot
ggplot(pca$rotation) +
  aes(x = PC2,
       y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```

